Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Program: Software Engineering

Operating Systems

Laboratory Work No. 7

Three Small Programs

Track A: The Real OS, from the Outside (Linux)

Elaborated by:
st. gr. FAF-243
Veacheslav Morari

Verified by:
Patricia Reţman

Chişinău – 2026

Contents

Introduction **2**

1 run.c **2**
1.1 Observations 2

2 zombie.c **3**
2.1 Observations 3

3 redirect.c **3**
3.1 Observations 4

4 pipeline.c **4**
4.1 Observations 5

Conclusion **5**

Introduction

The lecture on processes raised three questions: how a process is born, where `printf` writes, and how `ls | wc -l` works. The purpose of this laboratory work is to answer them with running code. Four small C programs are written and compiled on Linux: `run.c` shows `fork` + `exec` + `wait` at work, `zombie.c` makes a visible zombie process, `redirect.c` performs output redirection with `close` + `open` + `exec`, and `pipeline.c` builds a pipeline by hand with `pipe` + `fork` + `dup2`. The work was done on Ubuntu 26.04 using the system compiler `gcc`.

1. run.c

The program does what the shell does when the user types `ls -l`: it copies itself with `fork`, the copy becomes `ls` with `exec`, and the original waits for it with `waitpid`.

```
pid_t pid = fork();
if (pid < 0) { perror("fork"); exit(1); }

if (pid == 0) {
    printf("child: pid=%d, parent=%d\n", getpid(), getppid());
    execlp("ls", "ls", "-l", NULL);
    perror("exec");
    exit(1);
}

printf("parent: pid=%d, child=%d\n", getpid(), pid);
int status;
waitpid(pid, &status, 0);
if (WIFEXITED(status))
    printf("parent: child exited with %d\n", WEXITSTATUS(status));
```

After compiling with `gcc -Wall -o run run.c`, the run printed the expected lines:

```
$ ./run
parent: pid=4928, child=4929
child: pid=4929, parent=4928
total 96
drwxrwxr-x 2 veacheslav veacheslav  4096 Nov  2 15:31 Lab7
drwxrwxr-x 2 veacheslav veacheslav  4096 Nov  2 15:31 Lab8
-rw-rw-r-- 1 veacheslav veacheslav   562 Nov  7 20:43 run.c
-rwxrwxr-x 1 veacheslav veacheslav 16304 Nov  7 20:43 run
...
parent: child exited with 0
```

1.1. Observations

`fork()` returns the child's PID in the parent and 0 in the child, which is how each one knows which process it is. In my run the parent had PID 4928 and the child 4929, and the child's `getppid()` indeed printed 4928, confirming that the two processes are related

as parent and child. After `execlp` the child never comes back to the C code; the only line that would run after it is `perror("exec")`, and that runs only if `exec` failed. The parent's `waitpid` call is what keeps the child's exit status from being lost: without it, the child would stay as a zombie, which is the subject of the next part.

2. zombie.c

A zombie is a child that has ended but whose exit status has not yet been collected. The experiment makes one on purpose by adding `sleep(20);` just before `waitpid`, so that the parent delays the collection for 20 seconds while the child has already finished.

```
printf("parent: pid=%d, child=%d\n", getpid(), pid);
sleep(20);                /* <-- the only new line */
int status;
waitpid(pid, &status, 0);
```

The program was started in the background with `./zombie &`, and the process table was inspected two seconds later:

```
$ ./zombie & sleep 2 && ps -o pid,ppid,stat,cmd
[1] 4952
parent: pid=4952, child=4954
child: pid=4954, parent=4952
...listing...
    PID    PPID STAT CMD
   4833    4773 Ss   /usr/bin/bash
   4952    4833 S    ./zombie
   4954    4952 Z    [ls] <defunct>
   4957    4833 R+   ps -o pid,ppid,stat,cmd
```

The last line is the zombie: PID 4954, parent 4952, state `Z`, labelled `[ls] <defunct>`. After the 20 seconds elapsed the parent reached `waitpid`, the exit status was collected, and the zombie disappeared from the process table.

2.1. Observations

A zombie takes no CPU and no memory of its own, only a tiny slot in the kernel's process table so that the parent can still read the exit status. The parent's job is to call `wait` or `waitpid`. If the parent dies first, PID 1 (systemd on this system) adopts the child and reaps it, so an orphan does not stay as a zombie for long. The experiment also shows that a child can run to completion long before the parent notices, which is why real servers that fork many children often install a `SIGCHLD` handler to reap them as soon as they finish.

3. redirect.c

The program implements what the shell does for `ls -l > out.txt`: in the child it closes descriptor 1, opens the file (which, by the rule of the smallest free number, receives descriptor 1), then runs `ls`.

```
if (pid == 0) {
    close(1);
    int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); exit(1); }
    execlp("ls", "ls", "-l", NULL);
    perror("exec");
    exit(1);
}
waitpid(pid, NULL, 0);
```

Running the program printed nothing on the screen; the file `out.txt` contained the full `ls -l` listing:

```
$ ./redirect && cat out.txt
total 92
drwxrwxr-x 2 veacheslav veacheslav  4096 Nov  2 15:31 Lab7
drwxrwxr-x 2 veacheslav veacheslav  4096 Nov  2 15:31 Lab8
-rw-r--r-- 1 veacheslav veacheslav     0 Nov  7 20:45 out.txt
-rwxrwxr-x 1 veacheslav veacheslav 16208 Nov  7 20:45 redirect
-rw-rw-r-- 1 veacheslav veacheslav   477 Nov  7 20:45 redirect.c
...
```

3.1. Observations

`ls` has no idea that it is writing into a file. It writes to file descriptor 1 as always, except that descriptor 1 now points at `out.txt`. The redirection is set up entirely by the parent program before `exec`, which is exactly what the shell does when it sees `>`. This is why `fork` and `exec` are two separate calls in UNIX: the gap between them is where the child can reconfigure its own file descriptors without touching the program that is about to replace it.

The trick relies on the kernel rule that `open` always returns the smallest free descriptor. After `close(1)`, descriptor 1 is free; the next `open` call gets exactly it. If descriptor 0 had been closed instead, the file would have become standard input, not output.

4. pipeline.c

The program builds the pipeline `ls | wc -l` by hand: one `pipe`, two `fork`s. In `ls`, descriptor 1 becomes the writing end `p[1]`; in `wc`, descriptor 0 becomes the reading end `p[0]`. Both children close every pipe descriptor they do not need, and the parent closes both ends before waiting.

```
int p[2];
pipe(p);

pid_t ls = fork();
if (ls == 0) {
    close(p[0]);
    dup2(p[1], 1);
    close(p[1]);
```

```
    execlp("ls", "ls", NULL);
    perror("exec ls");
    exit(1);
}

pid_t wc = fork();
if (wc == 0) {
    close(p[1]);
    dup2(p[0], 0);
    close(p[0]);
    execlp("wc", "wc", "-l", NULL);
    perror("exec wc");
    exit(1);
}

close(p[0]);
close(p[1]);
waitpid(ls, NULL, 0);
waitpid(wc, NULL, 0);
```

The run printed `12`, the same number as `ls | wc -l` in the real shell:

```
$ ./pipeline && echo "--- real shell: ---" && ls | wc -l
12
--- real shell: ---
12
```

When the line `close(p[1]);` in the parent was commented out, `wc` never ended and the program hung until `Ctrl-C`.

4.1. Observations

A pipe is a kernel buffer with two ends: `p[1]` to write, `p[0]` to read. The `read()` call on the reading end returns 0 (EOF) only when every writing end is closed. If the parent still holds `p[1]` open, the kernel cannot deliver EOF, and `wc` waits forever. The rule of thumb is simple: after `fork` and `dup2`, close every pipe descriptor that the process does not plan to use.

The second observation is that the three processes — `ls`, `wc` and the parent — run concurrently. `ls` starts writing as soon as it has output; `wc` starts reading as soon as there is data in the pipe. If `ls` produces more than the pipe buffer can hold, the kernel pauses it until `wc` reads some data. This back-pressure is handled entirely by the kernel, and neither program needs to know about it.

Conclusion

The four programs showed what the shell actually does under the hood: `fork` + `exec` + `wait` to start a command, `close` + `open` between `fork` and `exec` for redirection, and `pipe` + two `fork`s + `dup2` + `close` for a pipeline. The idea that joins all three is the gap between `fork` and `exec`: the child inherits the parent's descriptors, can rearrange them freely, and only then becomes the new program. The experiments with the zombie and

the forgotten `close` made two common failure modes visible, which is useful preparation for debugging real multi-process programs.